Triggering Messages

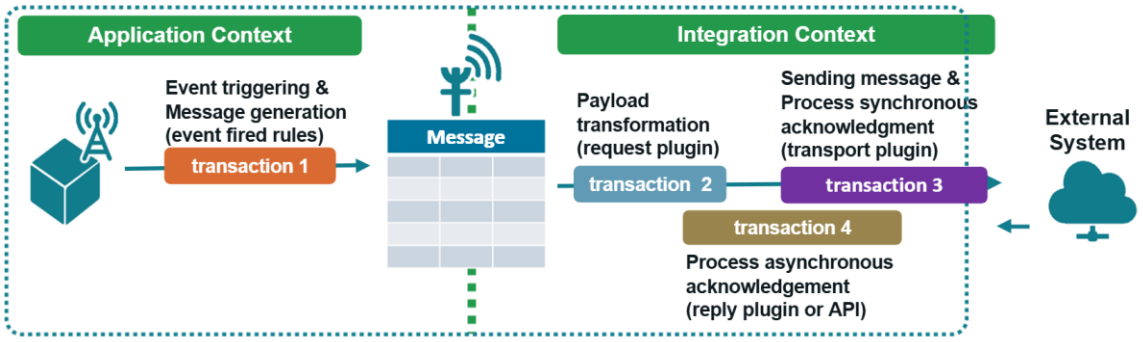
Use case – Bank account verification

Requirements	Design Approach
• Trigger a message when a bank account is added or modified • Create an abstract representation of an external system to define the key messaging components	1. Configure the bank account entity to trigger messages 2. Create a messaging destination 3. Deploy code changes

Use case – Bank account verification

The bank account message is triggered when a checking or savings bank account is added or modified. The message is created by the bank account verification event fired rules.

Guidewire servers and messaging transactions



- All activity from **event triggering** takes place in **transaction 1**
- Messaging occurs across multiple transactions
  - If an exception is thrown, work in the current transaction is rolled back

Guidewire servers and messaging transactions

The servers involved at each point of the process are important because they determine where payload work should occur. The majority of payload generation should be done in Event Fired rules on the **ui** role server that manages the given user's session. However, if some portion of payload generation must be done immediately before the message is sent, that portion of the payload can be generated on the **messaging** role server.

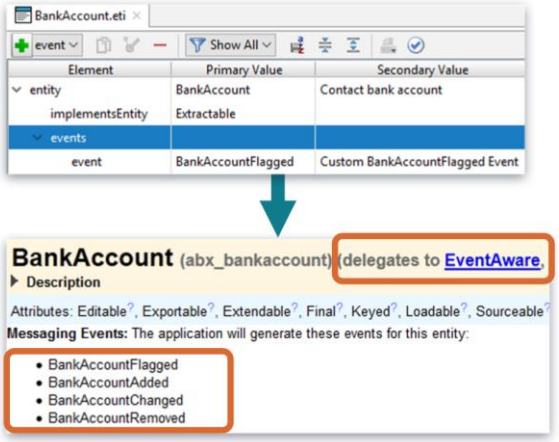
A message is always processed in at least two and as many as four transactions:

- Transaction 1 – Creates the message and generates the payload. This transaction always occurs.
- Transaction 2 – Prior to sending the message, an optional transaction may transform the message payload. This is usually done when the message must include information that can only be known immediately before sending the message.
- Transaction 3 – The sending of the message to the external system occurs in its own transaction. This transaction always occurs. If the external system acknowledges the message synchronously, then the acknowledgement is included in this transaction.
- Transaction 4 – If the external system acknowledges the message asynchronously, then a separate transaction processes the acknowledgement. This transaction does not occur for messages that are acknowledged synchronously.

Messaging Entities

EventAware entity

- A **messaging event** often occurs when adding, changing, or removing an instance of an EventAware entity
- An **EventAware** entity delegates to EventAware delegate
  - Configured by adding the **events** Element to the entity
    - Implements the default set of messaging events: Added, Changed and Removed



  - Naming convention
    - **<EventAwareEntityName><ChangeType>**
- Custom message events can be added
  - Example: Custom **BankAccountFlagged** message event added

EventAware entity

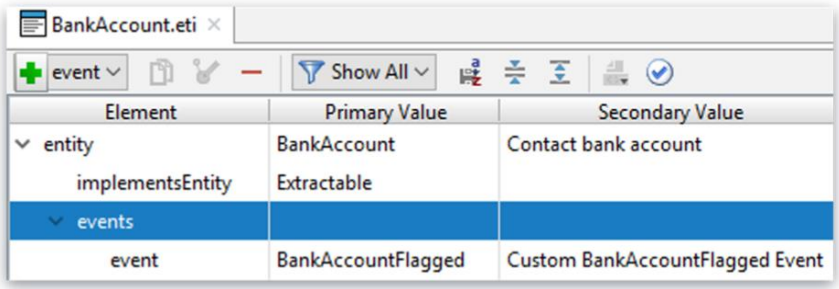
Most entities are persistent entities. Persistent entities describe specific information and store data in the database. An event-aware entity specifies an <events/> element in its XML definition which means that the entity delegates to the EventAware delegate. You cannot modify the set of the events associated with a base entity through extension. However, you can add additional events to a base entity through extension, even if that entity already contains a set of predefined events.

A delegate is an entity associated with an implemented interface that multiple parent entities can reuse. The EventAware entity is an internal, non-persistent entity that implements the messaging interface. For all entities that implement the EventAware delegate, the Guidewire application automatically creates specific events: entityNameChanged, entityNameRemoved, and entityNameAdded.
You can identify that an entity is EventAware by reviewing the entity in the Entity Editor in Guidewire Studio. You can also review the Data Dictionary to see the entity implements the EventAware delegate.

A messaging event typically occurs when an instance of an EventAware entity is created, changed, or deleted. Almost every EventAware entity has Added, Changed, and Removed events. The full list of events for each application is listed in the application's Integration Guide.

Data import tools such as ImportToolsAPI and import_tools command line tool are generic mechanisms for loading system data or sample data into the system and do not trigger messaging events for EventAware entities. Bulk data loaded using the table import tools also do not trigger messaging events. When loading business data that requires messaging events, do not use these tools, APIs, and mechanisms.

Triggering custom message events



- Useful when messaging must be initiated by something that cannot be captured in a standard (built-in) message event
- Syntax
  - **`eventAwareEntityInstance.addEvent("CustomEventName")`**
    - Only for EventAware entity
    - Triggered manually in code
  - Example: **`aBankAccount.addEvent("BankAccountFlagged")`**

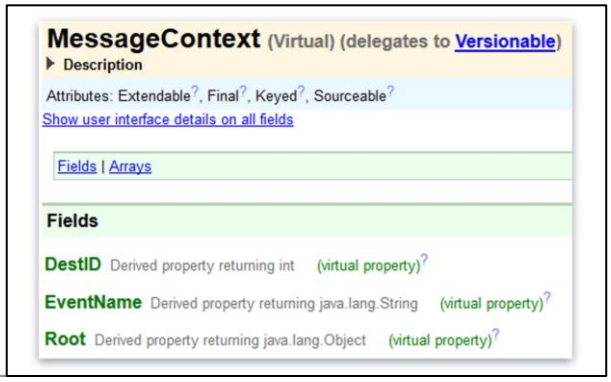
Triggering custom message events

Custom events provide additional flexibility in the messaging architecture. Sometimes, you may want to trigger a message event in a context where an EventAware object hasn't necessarily been created or changed. For example, you may want to provide the user with the ability to trigger messaging by clicking a button on the user interface. In some cases, you want to group messaging logic at a more granular level than EntityChanged.

Message events cannot be triggered for non-EventAware entities.

MessageContext entity

- An internal, **non-persistent**, virtual entity
  - Only used in **event fired** rules
- Important fields
  - **DestID**
    - Identifies the subscribing destination
  - **EventName**
    - Identifies the name of the triggering event



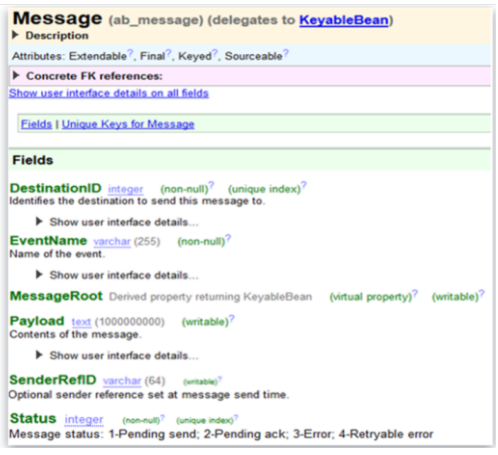
- **Root**
  - Can be used to reference data from the triggering entity
  - Example: `messageContext.Root typeis BankAccount`

MessageContext entity

Event Fired rules depend on the message context object. You can access this object using the messageContext variable. The messageContext object contains information such as the event name, messaging destination ID, and the Root object. Event fired rules often generate one or more messages, although the logic can omit creating messages as appropriate.

Message entity

- An entity containing message details and payload to be sent to an external system
  - **DestinationID**
    - Identifies the subscribing destination
  - **EventName**
    - Identifies the name of the triggering event
  - **MessageRoot**
    - Is a pointer to the triggering event-aware entity instance



- **Payload**
  - Holds the information to communicate
- **SenderRefID**
  - Is a sender reference number typically set at message send time
- **Status**
  - Message status pending and errored messages

Message entity

Message is a persistent entity that stores information about the message process in the Message database table. Users do not create message entities. Messages are created in event fired rules.
Avoid unnecessarily large or complex payloads if possible. A payload is information the information to be sent to an external system in response to an event.

xx_message database table
- **xx** represents the application code
- Example: ab_message (ab stands for addressbook)

MessageHistory entity

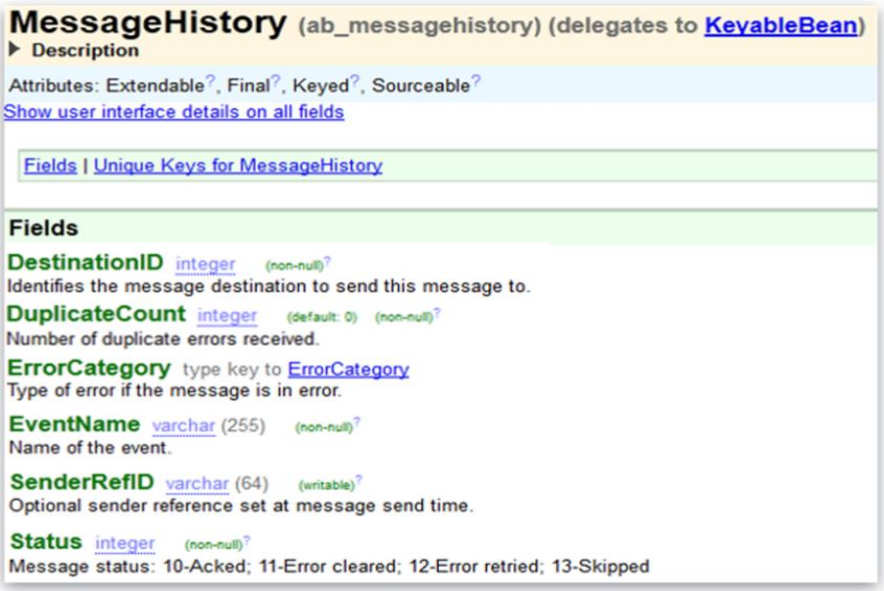
- An entity containing message history details
  - **DestinationID**
    - Identifies the subscribing destination
  - **DuplicateCount**
    - Number of duplicate message acknowledgments
  - **ErrorCategory**
    - Type of message processing error

MessageHistory entity

MessageHistory is a persistent entity that stores information about the message process in the MessageHistory database table.

xx_messagehistory database table
- **xx** represents the application code
- Example: ab_message (ab stands for addressbook)

- **EventName**
  - Identifies the name of the triggering event
- **SenderRefID**
  - Is a sender reference number typically set at message send time
- **Status**
  - Message status for acknowledged, skipped, or errored messages



MessageHistory (ab_messagehistory) (delegates to KeyableBean)

▶ Description

Attributes: Extendable?, Final?, Keyed?, Sourceable?

Show user interface details on all fields

Fields | Unique Keys for MessageHistory

Fields

DestinationID integer (non-null)?
Identifies the message destination to send this message to.

DuplicateCount integer (default: 0) (non-null)?
Number of duplicate errors received.

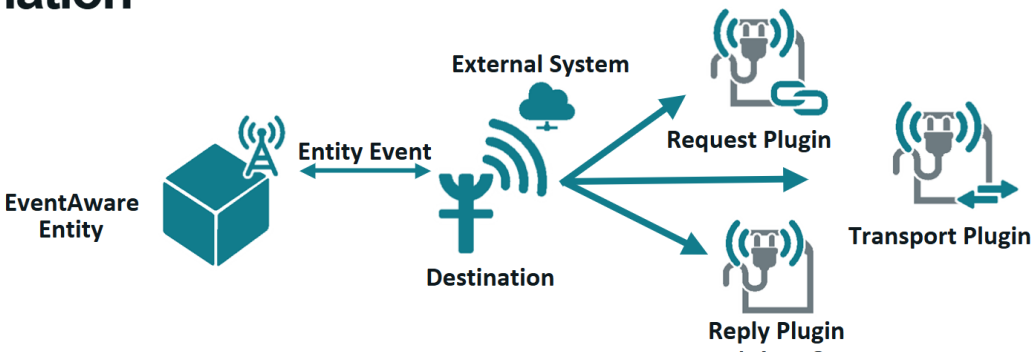
ErrorCategory type key to ErrorCategory
Type of error if the message is in error.

EventName varchar (255) (non-null)?
Name of the event.

SenderRefID varchar (64) (writable)?
Optional sender reference set at message send time.

Status integer (non-null)?
Message status: 10-Acked; 11-Error cleared; 12-Error retried; 13-Skipped

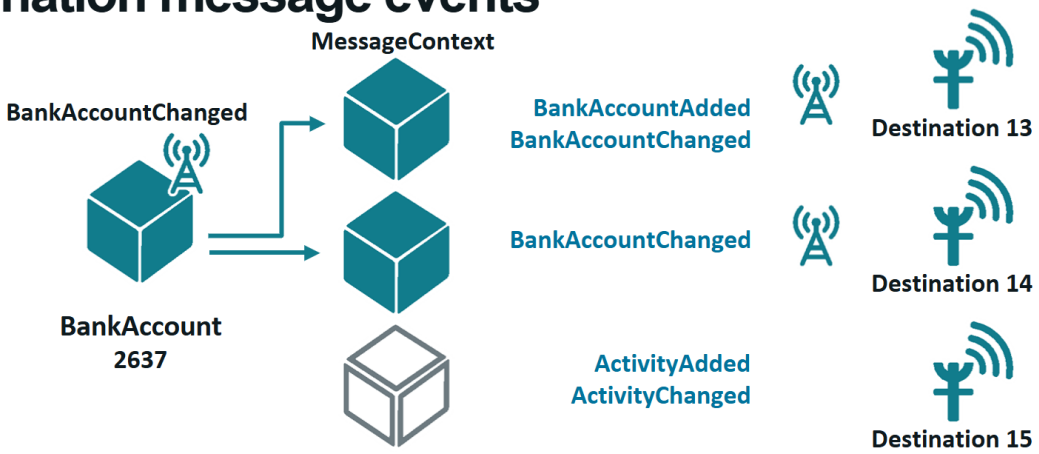
Destination



- A messaging **destination** is an abstraction of an external system
  - Subscribes to message events of interest
  - Defines list of message plugins used in processing outbound request messages and received inbound replies
  - Defines connectivity, retryable error parameters, predictable message ordering, multithreading, and distributed actions

Destination		
Each Guidewire message is sent to a single destination only. If there is a situation where multiple external systems need to receive the same information in response to a single event, then the Event Fired rules should create multiple messages, one for each destination of interest. (The term "multicasting" means that the same message is sent simultaneously to multiple receivers. Guidewire message does not make use of multicasting.)	Every message should receive a response. The response indicates to Guidewire that the message has been successfully processed. In some cases, the response comes immediately upon sending the message and simply indicates the message was successfully consumed. In other cases, the response comes sometime after the message is sent. The response can also contain data from the external system (such as a verification code or a report).	The mechanism for the external system response depends on if the message is synchronous or asynchronous. For synchronous replies, often the message transport plugin is the reply mechanism. For asynchronous replies, often a Guidewire Application published SOAP web service or a message reply plugin is the reply mechanism.

Destination message events



- Messaging destination subscribes to one or more specific message events
- When a message event occurs:
  - One MessageContext instance is created for each destination that listens to that event
  - Each MessageContext object is then processed by the event fired business rules

Destination message events

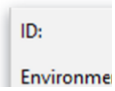
In the example above:
BankAccount 2637 is an instance of BankAccount, an event-aware entity
There are also three destinations

- Destination 13 listens for BankAccountAdded and BankAccountChanged events
- Destination 14 listens for the BankAccountChanged event
- Destination 15 listens for ActivityAdded and ActivityChanged events.

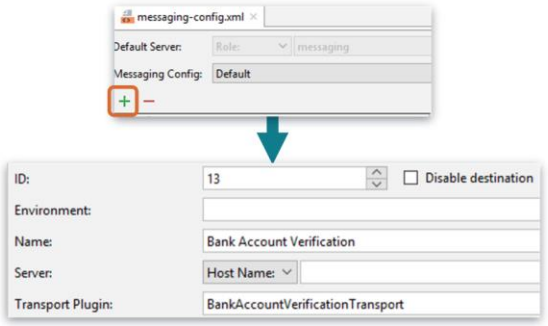
When BankAccount 2637 changes, a MessageContext object is created for destinations 13 and 14. Since destination 15 does not listen for BankAccountChanged events, no MessageContext object is created for destination 15.

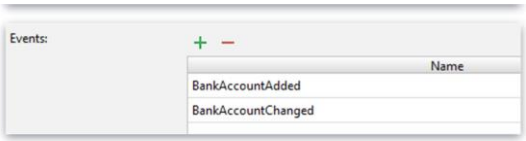
Defining a message destination

- Destinations are defined in the **messaging-config.xml** file
  - Under **configuration.config.Messaging**
- To create a new destination, click the plus symbol and define the **required fields**



- **ID**: unique ID value from 0 to 63
- **Name**: unique name displayed in the UI messaging administration screen
- **Transport Plugin**: sends message to external system(s)
  - Discussed in Sending Messages lesson
- **Events**: subscribed messaging events





Defining a message destination

ID – The destination ID must be an integer from 0 to 63, inclusive. Values greater than 63 are reserved by Guidewire.

Name – The destination name that is used in the application user interface. This can be a display key expression.

Transport Plugin – Is responsible for sending your message to the external system. The nature of this plugin will vary depending on the external system and/or the technology used to communicate to the external system (such as a JMS queue, a web service request to an external system, or an FTP call). Every destination must provide a message transport plugin implementation, but multiple destinations can use the same implementation

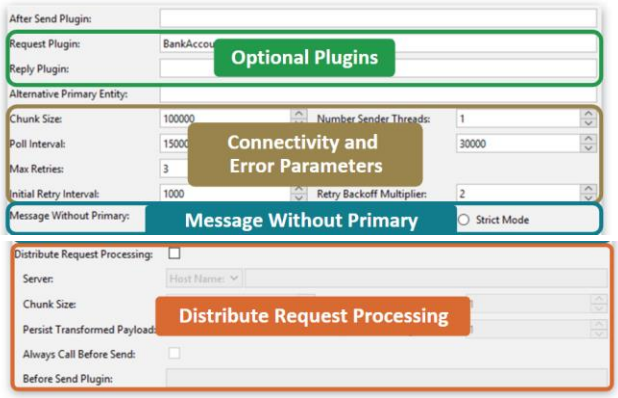
Events – A list of event names that correspond to the events relevant to this destination. For example, the BankAccountAdded and BankAccountChanged events might be of interest to this external system. Each event triggers the Event Fired rule set for that destination. One user action could trigger multiple events. If one action creates more than one event that the destination listens for, the application runs the Event Fired rules for every combination of event name and destination. The strings in the Events list are regex patterns. Developers must be careful to spell the regex patterns in the Events list correctly, as the destination will not work as expected if they are spelled incorrectly. For example, a destination that listens for ***BankAccountChange*** will not respond to a **BankAccountChanged** message event.

Best practice:

- For example, if the Events list included the string BankAccount, then the destination would listen for BankAccountAdded, BankAccountChanged, and BankAccountRemoved. In practice, though, it is **best** to **explicitly list all the events** a destination listens to.

Other messaging destination parameters

- Optional Plugins
  - Request plugin is discussed in the Message Payload Transformation lesson
  - Reply plugin is discussed in the Asynchronous Message Acknowledgements lesson
- Connectivity and error parameters
  - Discussed in the Sending Messages lesson
- Message Without Primary
  - Discussed in the Sending Messages lesson
- Distribute Request Processing
  - Discussed in the Master's Subscription and documentation



Other messaging destination parameters

Guidewire now offers the Master's Subscription with new training content for InsuranceSuite 8.0 and above. More than just advanced topics, the Master's Subscription includes a variety of eLearning videos designed for those who want to master their product knowledge and expertise, as well as deepen their understanding and application of best practices. New content is added monthly, providing continual learning and mastery.

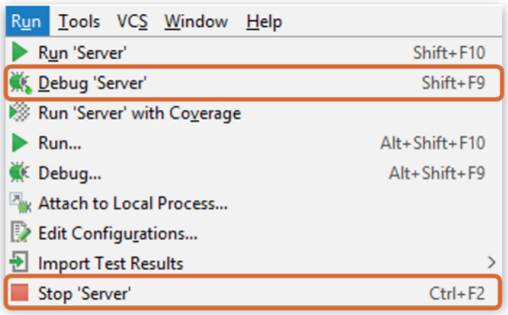
The Guidewire Master's Subscription is ideal for these roles:

- Developers who work on Guidewire products and desire a more in-depth understanding of the product and features
- System administrators who maintain Guidewire InsuranceSuite products, i.e., ongoing configuration or integration
- Anyone who supports their Guidewire Center of Excellence

Deploy the code changes

Restart Server

- Any new or modified destinations
  - Read only at server startup



Questions and Answers

1)

When does a message event get triggered?

Typically, a message event gets triggered when an event-aware entity is created, changed, or removed.

2)

What is the primary purpose of the MessageContext entity?

The primary purpose of the MessageContext entity is to create messages using information from the message event and the triggering entity.

3)

A Guidewire application has seven destinations and an event-aware entity called Escalation.

A) When an Escalation is created, what is the maximum number of MessageContexts that might get created?

B) What is the minimum?

A) Seven, which would occur if all seven destinations listen to EscalationAdded.

B) Zero, which would occur if none of the destinations listen to EscalationAdded.

4)

What plugin is required for every destination? What does this plugin do?

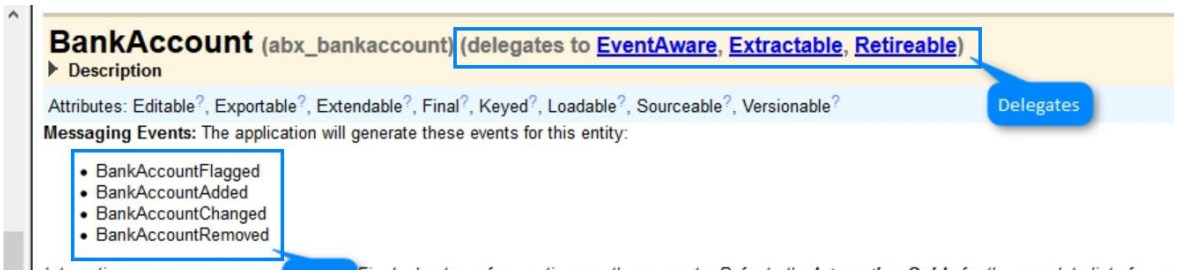
MessageTransport, which sends the message to the external system.

Demonstration

Examine Event Aware Entity and Destination

Bank Account Verification

- Check BankAccount.eti entity has event delegate



- Check the destination in the messaging-config.xml (ID can be numbers between 0 to 63)

